Haolin Shao 12413323
ADA(H)
Assignment 2

1) Description of Tarjan's DMST algorithm:

Tarjan's algorithm is divided into 2 process: contract & expand. The algorithm also assumes the graph is strongly connected (by adding O(n) edges with sufficiently high cost) and initialize by placing every edge (u, v) into the heap P[v] of its head vertex.

Contract:

The algorithm picks an arbitrary vertex A as starting point and keeps one active path of selected cheapest incoming edges $v_0(A) \leftarrow v_1 \leftarrow ... \leftarrow v_k$, where v_k is the current vertex being processed (v_i could be either an original vertex of the graph or a super-vertex obtained by contracting a circle of (super-)vertices). The algorithm will extract the cheapest incoming edge from a heap of all edges entering current v_k , let's call it (u,v). There are 3 cases with respect to u:

1. The edge (u,v) is a self-loop (in most situation u, v are from the same super-vertex), in which case it should be ignored and we should repeat above proocess until (u,v) isn't a self-loop.
2. Vertex u belongs to a (super-)vertex B contained in the path (possibly B=u). In this case a cycle is formed and it needs to be contracted into a super-vertex C. Every vertex on the cycle becomes a child of c, and we record that nesting. At the same time, all heaps of the cycle vertices are melded into the heap of c. After that, the cycle is treated as a single new vertex, and the search for a cheapest incoming edge continues from c.
3. Vertex u hasn't been added to the path, which means u doesn't belong to any super-vertices. Then we add u into our path and the search for a cheapest incoming edge continues from u.

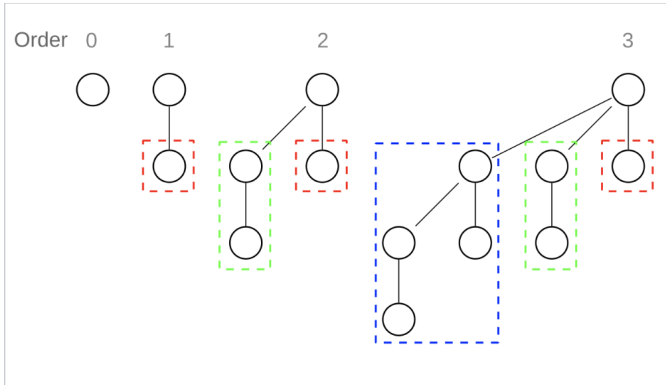
This continues until the entire graph has collapsed into one super-vertex.

And unlike Chu-Liu algorithm, the root is intentionally ignored during contraction. This is not a mistake: it keeps the selected edges arranged as a single path, and it means the same contraction phase can solve the problem for any root. Only after all contractions are finished does the algorithm start the root-dependent expansion step.

Expand:

The expansion phase reverses the recorded contractions. First, the algorithm dismantles the path from the chosen root r to the root of the contraction tree. That operation clears parent pointers for the sibling components that are not on the root path and pushes super-vertex among the sibling components into a work list.

Then the algorithm repeatedly removes a super-vertex from the work list. Suppose c is such a super-vertex and in[c] = (u, v) is its entering edge in the contracted solution. The edge (u, v) is assigned as the entering edge of v in the final arborescence, (we dismiss original incoming edge of v in the cycle) and v can be treated as a breakthrough point of the super-vertex. So the contractions below v are dismantled in the same way. Because the contraction tree stores the nesting of all cycles, the expansion step restores edges in the exact reverse order in which they were hidden.



Binomial trees of order 0 to 3: Each tree has a root node with subtrees of all lower ordered binomial trees, which have been highlighted. For example, the order 3 binomial tree is connected to an order 2, 1, and 0 (highlighted as blue, green and red respectively) binomial tree.

Figure 1: Binomial trees

2) Data structure:

Besides common data structures, we use meldable heap (specifically binomial heap or leftist tree) to store the edges so that we can quickly extract the cheapest incoming edge of all edges entering a (super-)vertex and meld 2 heaps. We also use disjoint-set union to record the super-vertex into which a vertex was contracted in order to check whether a cycle is formed or not fast.

1. Meldable heap:

   We take binomial heap as an example. Before we talk about binomial heap, we need to introduce binomial tree first, which are defined recursively as follows (Figure 1):

   - A binomial tree of order 0 is a single node.
   - A binomial tree of order k has a root node whose children are roots of binomial trees of orders k-1, k-2, ..., 2, 1, 0 (in this order).

   A binomial heap is implemented as a set of binomial trees that satisfy the binomial heap properties:

   - Each binomial tree in a heap obeys the minimum-heap property: the key of a node is greater than or equal to the key of its parent.
   - There can be at most one binomial tree for each order, including zero order.

   The first property ensures that the root of each binomial tree contains the smallest key in the tree. It follows that the smallest key in the entire heap is one of the roots.

   The second property implies that a binomial heap with n nodes consists of at most $1 + log_2 n$ binomial trees. The number and orders of these trees are uniquely determined by the number of nodes n. There is one binomial tree for each nonzero bit in the binary representation of the number n. For example, the decimal number 13 is 1101 in binary, and thus a binomial heap with 13 nodes will consist of three binomial trees of orders 3, 2, and 0 (see figure 2).

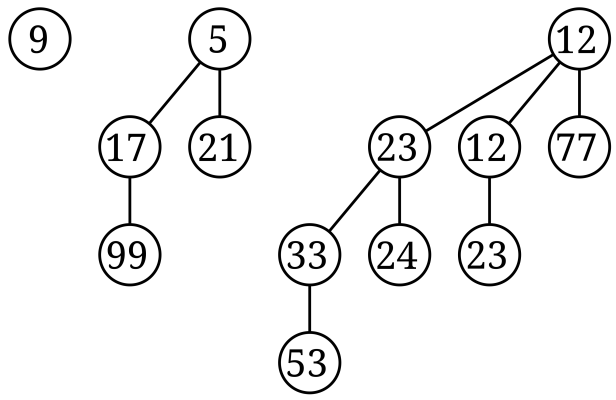


Figure 2: A binomial heap with 13 nodes

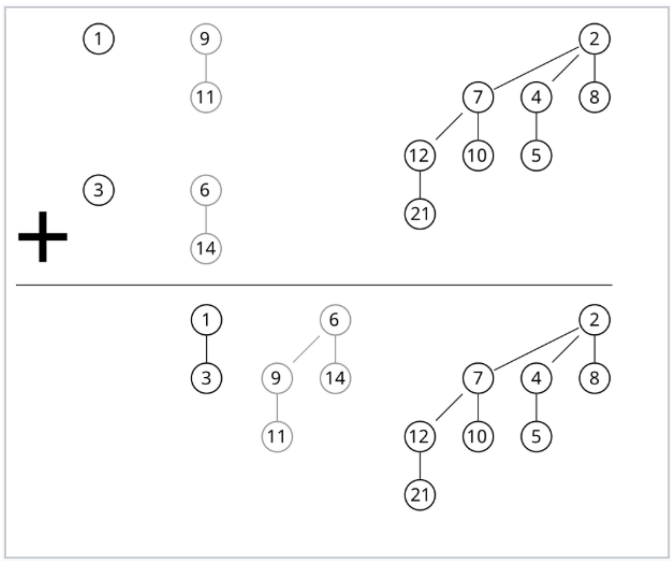


Figure 3: meld two binomial heaps

MELD: Melding two heaps is done much like adding binary numbers: we merge the root lists by order, and whenever two trees of the same order j appear, we link them by making the larger root a child of the smaller root. The two trees are merged to one tree of order j+1. It may later become necessary to merge this tree with some other tree of order j+1 in one of the two input heaps.Since there are only O(logn) different orders, this takes O(logn) time. (Figure 3)

INSERT: Inserting a new element to a heap can be done by simply creating a new heap containing only this element and then merging it with the original heap. Because of the merge, a single insertion takes time $O(\log n)$. However, this can be sped up using a merge procedure that shortcuts the merge after it reaches a point where only one of the merged heaps has trees of larger order. With this speedup, across a series of k consecutive insertions, the total time for the insertions is $O(k + \log n)$. Another way of stating this is that (after logarithmic overhead for the first insertion in a sequence) each successive insert has an amortized time of $O(1)$ (i.e. constant) per insertion.

EXTRACT MIN: To extract the minimum element from the heap, first find this element (O(logn)), remove it from the root of its binomial tree, and obtain a list of its child subtrees (which are each themselves binomial trees, of distinct orders). Transform this list of subtrees into a separate binomial heap by reordering them from smallest to largest order. Then merge this heap with the original heap. Since each root has at most $\log_2 n$ children, creating this new heap takes time $O(\log n)$. Merging heaps takes time $O(\log n)$, so the entire delete minimum operation takes time $O(\log n)$.

LAZY TAG: In contraction, we need to update "const[a] = -w[in[a]]". If we do it one by one, the time complexity would be linear. In order to ensure the overall time complexity is O(mlogn), we need to update in O(logm). To efficiently adjust edge weights after selecting the minimum incoming edge, we augment a binary meldable heap (e.g., a Leftist Tree) with a lazy propagation mechanism.
We define the true cost of an edge as: True Weight = node.key + node.lazyTag (where lazyTag is initially 0). Instead of explicitly updating all edge weights, a global offset is simply added to the root's lazyTag in O(1) time.To preserve the correctness of heap operations, we enforce a push-down mechanism. Before any operation (such as meld or extract-min) visits a node x, we propagate its tag to its children:

- PUSH DOWN:

```
x.key += x.lazy_tag;
if (x.left) x.left.lazy_tag += x.lazy_tag;
if (x.right) x.right.lazy_tag += x.lazy_tag;
x.lazy_tag = 0;
```

Improvement over the Lecture Notes:
The lecture notes suggest calculating edge weights dynamically by traversing a compressed disjoint-set forest (Figure 5). That approach heavily couples cycle detection with weight maintenance. By utilizing lazy tags inside the heap, we effectively decouple these two tasks: the disjoint-set is solely responsible for O(1) cycle detection, while the heap independently manages weight updates. This separation of concerns makes the algorithm significantly clearer and easier to implement.

2. Disjoint-set union:

A standard union-find structure stores a parent pointer for each element. Initially each pointer points at NULL. As contraction goes, when a circle formulated, all the (super-)vertices' pointers will point at the new super-vertex. There are 2 operations it use.

FIND: The find operation follows parent pointers until it reaches a root, which is the representative of the set. This happens when we check whether a cycle is formed or not. To make future queries faster, "path compression" is applied: every vertex visited during find is redirected to the root representative.

UNION: The operation Union(x, y) replaces the set containing x and the set containing y with their union. Union first uses Find to determine the roots of the trees containing x and y. If the roots are the same, there is nothing more to do. Otherwise, the two

trees must be merged. This is done by either setting the parent pointer of x's root to y's, or setting the parent pointer of y's root to x's and the order matters. In an efficient implementation, tree height is controlled using union by size, which is simply its number of descendants (including the node itself) or union by rank, which is an upper bound for its height. When two sets are merged, we attach the smaller tree under the larger one and update information stored in the root node, keeping the structure shallow.

With both optimizations, a long sequence of operations is essentially linear time, more precisely $O((m+n)\alpha(n))$, where α is the inverse Ackermann function.

3) **Complexity:**

Time complexity: $O(M \log N)$.

Step	Cost per operation	How many times	Total
Initial heap inserts	O(log n) with a binomial heap	m edges	O(m log n)
extract-min calls	O(log n)	At most once per edge, plus failed removals of self-loops	O(m log n)
meld operations	O(log n)	At most one per contraction, hence O(n) times	O(n log n)
find on endpoints	Amortized near-constant with path compression	O(m + n) calls in total	dominated by heap costs

Note on Heap Complexity: The maximum number of elements in any meldable heap at any time is bounded by the total number of edges m. Thus, a strict bound for a heap operation is $O(\log m)$. However, since the graph can have at most $O(n^2)$ edges, we have $\log m \leq \log(n^2) = 2\log n$. Therefore, $O(\log m)$ is asymptotically equivalent to $O(\log n)$. And a strongly connected directed graph on n vertices has at least n edges, so $O(n \log n)$ is absorbed by $O(m \log n)$. This guarantees that the total cost for heap operations is exactly $O(m \log n)$.

Space complexity: $O(N+M)$.
The input graph itself stores m edges. Each edge appears in exactly one heap node, so the heap storage is O(m). The vertex arrays in, const, prev, parent, and children use O(n) space for the original vertices, plus O(n) more for the at-most n - 1 super-vertices created by contractions. The union-find or path-compression structure also uses linear space. Hence the total auxiliary space is O(m + n).

4) **Summary:**
The algorithm is best understood as a controlled process of repeatedly selecting cheapest incoming edges, compressing any cycle that appears, and remembering exactly how to undo that compression later. Its efficiency comes from two design choices: keep the incoming edges in meldable heaps, and keep the contraction history in a tree so that weights and endpoints can be interpreted lazily instead of being rewritten after every contraction. That is why the implementation achieves O(m log n) time with only linear extra space.